

Linked List Operations: Insert, Delete & Reverse

CSCI 235 — Class #8

Tong Yi

Hunter College, CUNY

Thursday, September 24, 2026

Where We Are

- Tuesday: an **ADT** is an interface; a **node** is one piece of data plus the address of the next; head points at the first node and `nullptr` ends the chain.
- You can now **traverse** a list — print it, count it, search it, destroy it.
- Every one of those operations only *reads* the links.

Today: writing the links

Insert, delete, and reverse all change the shape of the chain. The code is short — three or four lines each — but the **order** of those lines decides whether you end up with a correct list, a leaked list, or a crash.

The Rule Behind Everything Today

Pointer surgery, in one sentence

Before you overwrite any pointer, make sure something else still holds the address you are about to lose.

An analogy: the only key

You are holding the only key to a room. Someone hands you a different key and you put it in the same pocket. The first key is gone — the room still exists, still locked, and now nobody can ever get in. That is a leaked node, and it happens in exactly one assignment statement.

You will see this same rule repeatedly today, in different disguises.

Prediction 1: What Does This Bug Create?

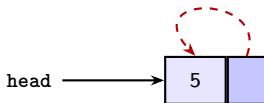
Start with head pointing to $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{nullptr}$, then run:

```
Node* newNode = new Node{5, nullptr};  
head = newNode;  
newNode->next = head;
```

- ☐ A $5 \rightarrow 10 \rightarrow 20 \rightarrow 30$
- ☐ B A one-node self-cycle at 5, while the old list is leaked
- ☐ C $5 \rightarrow \text{nullptr}$, with the old list deleted
- ☐ D A compile-time error because head changes twice

Vote on your phone: scan the QR code or use the Brightspace link.

B. A self-cycle and a leak



old `10 → 20 → 30` chain
still allocated, now unreachable

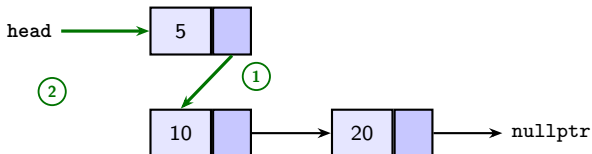
Why

Moving `head` first destroys the only route to the old first node. The following assignment reads the new value of `head`, so the new node stores its own address in `next`.

Prepend: Insert at the Front

Two steps: point the new node at the old first node, then move head.

```
void prepend(Node*& head, int value) {  
    Node* newNode = new Node{value, head}; // next = OLD first node  
    head = newNode;                        // now head points to it  
}
```

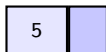


$O(1)$ — no matter how long the list is

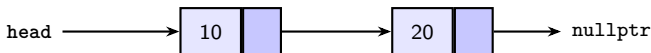
Compare with `MiniVector`: inserting at index 0 shifts all n elements. Here nothing moves; two pointers are written. This demonstrates one of the main advantages of linked lists.

Prepend Animation: Preserve, Then Move

newNode

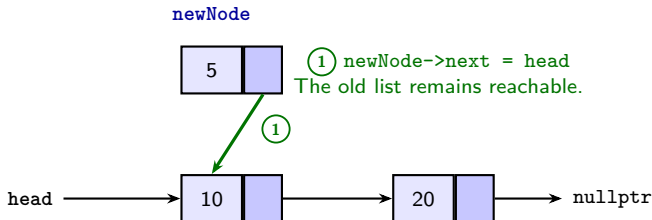


Step 0: allocate node 5
Its next is not connected yet.



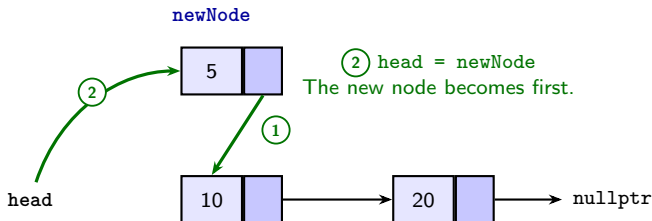
Click once per pointer write. The address of node 10 is preserved before head moves.

Prepend Animation: Preserve, Then Move



Click once per pointer write. The address of node 10 is preserved before head moves.

Prepend Animation: Preserve, Then Move



Click once per pointer write. The address of node 10 is preserved before head moves.

Prediction 2: Correct Prepend Ordering

Which ordering safely prepends `newNode`?

- ☐ A `head = newNode; then newNode->next = head;`
- ☐ B `newNode->next = head; then head = newNode;`
- ☐ C `delete head; then head = newNode;`
- ☐ D `head->next = newNode; then head = newNode;`

Vote on your phone: scan the QR code or use the Brightspace link.

B. Connect first, then move head

Teaching rationale

`newNode->next = head`; copies the old first node's address into a safe place. Only then can `head` change without losing the original chain.

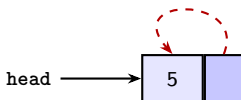
General rule

Preserve an address before overwriting the pointer that holds it.

The Classic Prepend Bug

The same two statements, in the wrong order:

```
Node* newNode = new Node{value, nullptr};  
head = newNode;           // (1) head now points at the NEW node  
newNode->next = head;     // (2) ... which now points at ITSELF
```



the rest of the list
is unreachable — and
next points at itself

Two bugs for the price of one

Every node after the first is **leaked** (nothing points to them), and the list now has a **cycle**, so any traversal loops forever — `curr != nullptr` never becomes false.

Append: Insert at the Back — Your Turn

The new node goes at the *end*. Unlike *prepend*, nothing already points there, so you have to go find it. **Two steps. Write them.**

```
void append(Node*& head, int value) {  
    Node* newNode = new Node{value, nullptr};  
  
    STEP 1    // what if the list is empty?  
  
    STEP 2    // otherwise: find the last node,  
              // then hook newNode on  
}
```

Before you start writing

Ask which pointer each step has to change. In one of them it is `head` itself; in the other it is some node's `next`. That is the whole difference between the two cases. Step 2 needs the loop that stops *on* the last node, not the one that walks past it.

Append: Insert at the Back — Your Turn

The new node goes at the *end*. Unlike prepend, nothing already points there, so you have to go find it. **Two steps. Write them.**

```
void append(Node*& head, int value) {  
    Node* newNode = new Node{value, nullptr};  
  
    if (head == nullptr) {           // STEP 1: empty list  
        head = newNode;             // the new node IS the list  
        return;  
    }  
  
    STEP 2    // still yours: find the last node,  
              // then hook newNode on  
}
```

Step 1, and why it is a separate case

An empty list has no node whose next could be set — head itself is what must change. Exactly the situation from `insertAt(0, ...)`.

Append: Insert at the Back — Your Turn

The new node goes at the *end*. Unlike *prepend*, nothing already points there, so you have to go find it. **Two steps. Write them.**

```
void append(Node*& head, int value) {
    Node* newNode = new Node{value, nullptr};

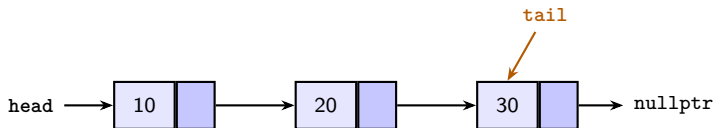
    if (head == nullptr) {           // STEP 1: empty list
        head = newNode;
        return;
    }

    Node* curr = head;               // STEP 2: walk to the end
    while (curr->next != nullptr) { // stop ON the last node
        curr = curr->next;
    }
    curr->next = newNode;             // hook it on
}
```

Why the empty check cannot be skipped

Delete Step 1 and run it on an empty list: `curr` starts as `nullptr`, so `curr->next` on the very first test dereferences a null pointer and the program dies.

Append Is $O(n)$ — Unless You Keep a Tail



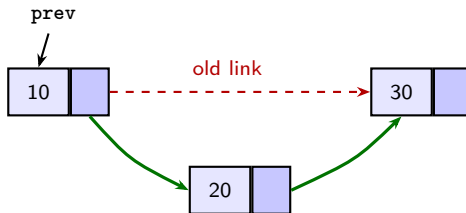
A second data member, `tail_`, pointing at the last node when the list is non-empty, turns append into $O(1)$ — no walk at all.

The price of a shortcut

`tail_` is now a second invariant you must maintain in *every* operation: prepend into an empty list must set it, and removing the last node requires finding its predecessor, which in a singly linked list still takes $O(n)$. Every shortcut in a data structure is paid for somewhere.

Insert in the Middle: You Need the *Predecessor*

To place a node between two existing nodes, the pointer you must change lives in the node **before** the insertion point.



Two writes: `newNode->next = prev->next;` then `prev->next = newNode;`

Order, again

Do `prev->next = newNode;` first and the address of node 30 is gone — `newNode->next` would then read the value you just overwrote, and the tail of the list is leaked.

Insert at a Position, Completely

Walk prev and curr together. When the walk stops, curr is the node the new one must point at, and prev is the node that must point at the new one.

```
// Insert value so that it ends up at index pos (0-based).
bool insertAt(Node*& head, int pos, int value) {
    Node* prev = nullptr;
    int i = 0;
    Node* curr = head;    // curr must still be valid after the loop

    while (i < pos && curr != nullptr) {
        i++;
        prev = curr;
        curr = curr->next;
    }

    if (i == pos) {        // we got all the way there
        Node* newNode = new Node{value, curr}; // 1 point at the successor
        if (prev == nullptr) {
            head = newNode;           // 2a no predecessor: head
        } else {
            prev->next = newNode;      // 2b otherwise: prev->next
        }
        return true;
    }

    return false;         // ran off the end, or pos was negative
}
```

Why Position 0 Needs No Special Case

The front is not a separate case here, only a separate *destination*.

`prev == nullptr` is “there is no predecessor”

Inserting at the front is the one position with no node in front of it. Instead of testing `pos == 0` up front, this version simply never moves `prev` off `nullptr` — so the test that asks “where do I write?” also answers “is this the front?”

Three more things it gets for free

- **Negative pos:** the loop never runs, `i` stays 0, and `0 ≠ pos` — rejected with no explicit check.
- `pos == length:` the walk ends with `curr == nullptr` and `i == pos`, so `new Node{value, nullptr}` appends. No append branch.
- **No leak on refusal:** nothing is allocated until `if (i == pos)` has said yes.

And why `i == pos` rather than `curr != nullptr`? The loop can stop for two different reasons; counting the steps is what tells you which one.

Prediction 3: What Happens After delete?

Assume the list is non-empty.

```
delete head;  
head = head->next;
```

- A It works because next remains inside the old node
- B It leaks only the first node
- C It reads through a dangling pointer: use-after-free
- D It always causes a double delete

Vote on your phone: scan the QR code or use the Brightspace link.

C. Use-after-free

Why

After `delete head;`, `head` still contains an address, but the object at that address no longer belongs to the program. Evaluating `head->next` is an invalid read.

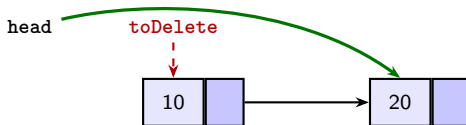
Safe order

Save the node you are about to remove, advance `head` while that node is still alive, then delete the saved address.

Delete the First Node

Same shape as the `clear()` loop from Tuesday — save, move, free:

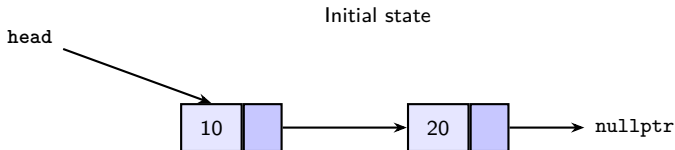
```
bool removeFront(Node*& head) {  
    if (head == nullptr) { return false; } // nothing to remove  
    Node* toDelete = head; // 1. remember the node to free  
    head = head->next;      // 2. unhook it from the list  
    delete toDelete;       // 3. NOW free it  
    return true;  
}
```



The two ways to get this wrong

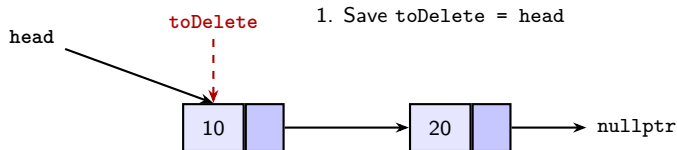
`delete head; first, then head = head->next;` — a read from freed memory. Or `head = head->next;` with no `toDelete` — the first node is leaked, silently, forever.

Remove Front Animation: Save, Advance, Delete



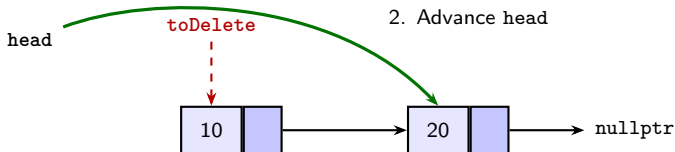
The node stays alive until its `next` address has been used.

Remove Front Animation: Save, Advance, Delete



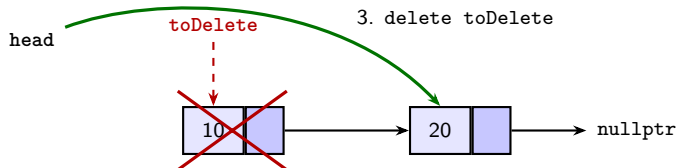
The node stays alive until its next address has been used.

Remove Front Animation: Save, Advance, Delete



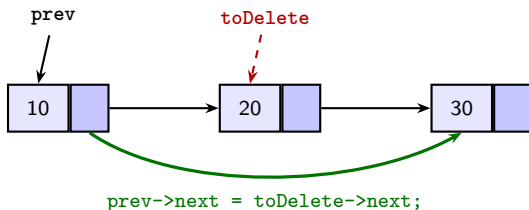
The node stays alive until its next address has been used.

Remove Front Animation: Save, Advance, Delete



The node stays alive until its next address has been used.

Delete in the Middle: Bypass, Then Free



Bypass the node first, so the list is intact and correct, and **only then** delete `toDelete`;

You cannot reach the predecessor by following links from the node itself

Given only a pointer to the node itself, a singly linked list provides no link back to its predecessor. If you also have head, you can search from the beginning — which takes $O(n)$. This is exactly why a prev pointer, or a doubly linked list, exists.

Delete by Value, Completely

The same walk as insertAt — stop on the node you want, with prev one step behind it.

```
bool removeValue(Node*& head, int target) {
    Node* prev = nullptr;
    Node* curr = head;

    while (curr != nullptr && curr->data != target) {
        prev = curr;
        curr = curr->next;
    }

    if (curr == nullptr) { // target is not in the list
        return false;
    }

    if (prev == nullptr) { // removing the first node
        head = curr->next; // 1 unhook: head moves on
    }
    else {
        prev->next = curr->next; // 1 unhook: bypass curr
    }
    delete curr;           // 2 then free -- curr still holds it

    return true;
}
```

Two Functions, One Shape

`insertAt` and `removeValue` follow the same general pattern: walk with `prev` and `curr`, then update either `head` or `prev->next`.

	<code>insertAt</code>	<code>removeValue</code>
Walk	<code>until i == pos</code>	<code>until curr->data == target</code>
Failed?	<code>i != pos $\Rightarrow$ false</code>	<code>curr == nullptr $\Rightarrow$ false</code>
No prev	<code>write head</code>	<code>write head</code>
Has a prev	<code>write prev->next</code>	<code>write prev->next</code>

Why the loop tests `curr->data`

Because `curr` *is* the candidate. No need to stop one node early and peer forward — `prev` already trails behind, holding the pointer you will rewrite. One walk, two useful pointers.

When Do You Need an Extra Pointer?

Only when the pointer you are about to overwrite is your *only* route to the node.

removeValue: no extra pointer

```
prev->next = curr->next;  
delete curr;
```

After the unhook, `curr` still holds the node's address — it was never overwritten. Nothing to save.

removeFront: extra pointer needed

```
Node* toDelete = head;  
head = head->next;  
delete toDelete;
```

Here `head` *is* the only pointer to the first node, and the unhook overwrites it. Save first.

The rule, one more time

Do not save a pointer out of habit — save it when the write you are about to make would destroy your last route to something you still need. Then unhook, and only then delete.

The Edge Cases, Collected

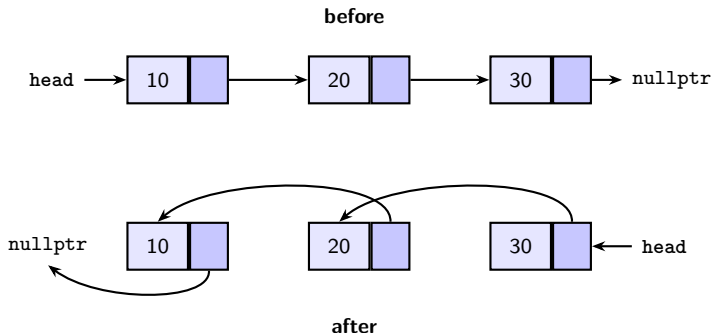
Situation	What is special	If you forget
Empty list	no node exists; head is what changes	null dereference (crash)
Insert at position 0 / delete the first node	no prev node exists	crash, or the whole list leaked
One-node list	first node is also the last	if maintaining tail_, it may be left dangling
Delete the last node	the predecessor's next must become nullptr	the predecessor may still point at freed memory
Value not present	the loop runs off the end	dereferencing nullptr

Test these five, every time

This is the Day 6 lesson in a new setting: bugs live at the boundaries. A linked-list function that works on a three-node list in the middle tells you almost nothing.

Reversing a List, In Place

Reverse means: make every next point the other way, and make head point at what used to be the last node — **without** allocating a single new node.



The nodes and their data never move. Only the links change — which is the whole point: a linked list's *order* lives in its pointers, not in where the elements sit.

Three Pointers Are Exactly Enough

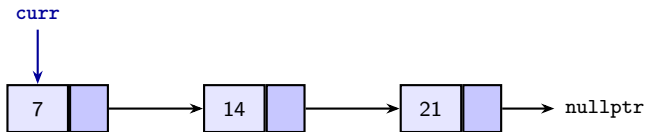
```
void reverse(Node*& head) {  
    Node* prev = nullptr;           // the part already reversed  
    Node* curr = head;              // the node being flipped now  
    while (curr != nullptr) {  
        Node* succ = curr->next;    // 1. SAVE -- we are about to lose it  
        curr->next = prev;          // 2. FLIP this node's link backwards  
        prev = curr;               // 3. the reversed part grew by one  
        curr = succ;              // 4. move on  
    }  
    head = prev;                   // prev is the old LAST node  
}
```

Why succ must be saved first

Line 2 overwrites `curr->next` — the only record of where the rest of the list is. Without line 1, everything after `curr` becomes unreachable on the very first iteration. This is the “only key” rule again.

The variable is called `succ` (successor), not `next`, so that it is never confused with the `next` field inside a node. They are different things: one is a pointer you own, the other lives in the node.

Reverse Animation: One Node, Four Pointer Actions

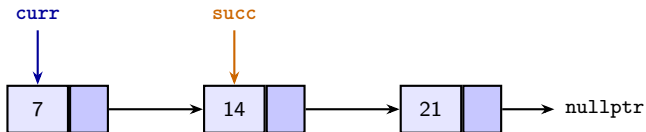


`prev = null`

Initial: `prev = null, curr = 7`

Advance one overlay at a time: save succ, flip, advance prev,
advance curr.

Reverse Animation: One Node, Four Pointer Actions

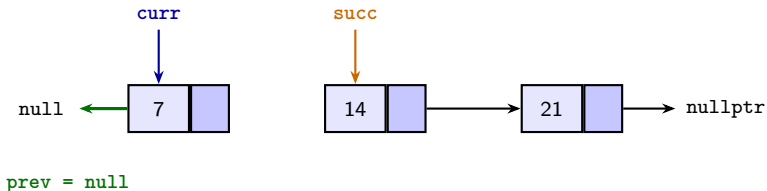


```
prev = null
```

1. SAVE: `succ = curr->next`

Advance one overlay at a time: save succ, flip, advance prev, advance curr.

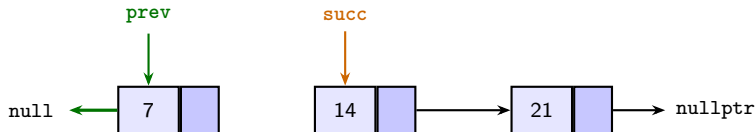
Reverse Animation: One Node, Four Pointer Actions



2. FLIP: $\text{curr} \rightarrow \text{next} = \text{prev}$

Advance one overlay at a time: save succ, flip, advance prev, advance curr.

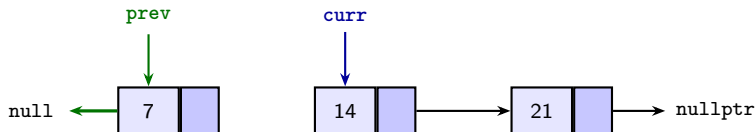
Reverse Animation: One Node, Four Pointer Actions



3. ADVANCE: `prev = curr`

Advance one overlay at a time: save `succ`, flip, advance `prev`,
advance `curr`.

Reverse Animation: One Node, Four Pointer Actions



4. ADVANCE: `curr = succ`

Advance one overlay at a time: save succ, flip, advance prev, advance curr.

Tracing reverse() on $10 \rightarrow 20 \rightarrow 30$

End of iteration	succ saved	prev after	curr after	list so far
1	20	10	20	$10 \rightarrow \text{null}$
2	30	20	30	$20 \rightarrow 10 \rightarrow \text{null}$
3	null	30	null	$30 \rightarrow 20 \rightarrow 10 \rightarrow \text{null}$

the loop ends, and $\text{head} = \text{prev} = \text{node}(30)$

Read the invariant down the table

After every iteration, `prev` is the head of a correctly reversed list and `curr` is the head of the untouched remainder. The loop moves one node from the second list to the first. When the remainder is empty, we are done.

Prediction 4: Pointer State After Two Iterations

For $7 \rightarrow 14 \rightarrow 21 \rightarrow \text{nullptr}$: after **two complete iterations**, where do `prev` and `curr` point?

- ☐ A `prev` $\rightarrow$ 7, `curr` $\rightarrow$ 14
- ☐ B `prev` $\rightarrow$ 14, `curr` $\rightarrow$ 21
- ☐ C `prev` $\rightarrow$ 21, `curr` $\rightarrow$ null
- ☐ D `prev` $\rightarrow$ 14, `curr` $\rightarrow$ 7

Vote on your phone: scan the QR code or use the Brightspace link.

Prediction 4: Reveal

B. `prev` $\rightarrow$ 14, `curr` $\rightarrow$ 21

Teaching rationale

Two nodes have moved into the reversed prefix, so `prev` points at 14. The untouched remainder starts at 21, so that is where `curr` stands, ready to be flipped next.

Why the question does not ask about `succ`

`succ` is declared *inside* the loop body, so it does not exist between iterations — a fresh one is created and destroyed on every pass. Asking where it points “after” an iteration would be asking about a variable that is already gone.

Prediction 5: What If succ Is Not Saved?

Suppose the loop body is reduced to:

```
curr->next = prev;  
prev = curr;  
curr = curr->next;
```

On the first iteration, where does curr go?

- ☐ A To the old second node
- ☐ B To the old last node
- ☐ C To nullptr, ending the loop and leaking the remainder
- ☐ D It stays on the first node forever

Vote on your phone: scan the QR code or use the Brightspace link.

C. curr becomes nullptr

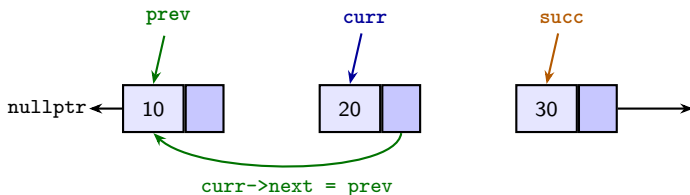
Why it fails on iteration 1

Initially, `prev` is `nullptr`. The flip stores `nullptr` into `curr->next`. The later expression `curr = curr->next;` therefore follows the new backward link, not the old forward link. Nodes 14 and 21 become unreachable.

Purpose of `succ`

`succ` preserves the entrance to the untouched remainder before the forward link is overwritten.

The Reversal, Step by Step



The snapshot just after iteration 2's flip: `prev` and `curr` have not advanced yet. Node 20 now points backwards at node 10, node 10 points at `nullptr`, and `succ` is holding node 30 so the rest of the list is still reachable.

Why Reversal Is $O(n)$ Time and $O(1)$ Space

- **Time $O(n)$:** the loop body is a fixed number of pointer writes, and it runs once per node.
- **Space $O(1)$:** three pointer variables, whatever the length of the list. Nothing is allocated, nothing is copied.

Compare: reversing a MiniVector

You would swap `data_[i]` with `data_[n-1-i]` — also $O(n)$ time and $O(1)$ auxiliary space, but it **rearranges the elements themselves**. A linked list reverses its order by changing links; the nodes themselves do not move.

Where These Three Operations Are Going

Today's operation	Where it reappears
Insert / remove at the front, $O(1)$	Day 9, Stack: push and pop are literally prepend and remove-front
Append with a tail pointer	Day 10, Queue: enqueue at the back, dequeue at the front, both $O(1)$
Building a chain backwards	Day 12, BFS: reconstructing the maze path by following each cell to its predecessor — which produces the path <i>reversed</i>

A promise about Project 2

You will not be asked to invent anything new for the path reconstruction in P2. It is today's reversal, applied to a chain you built while searching.

Your Questions From Tuesday

Thank you for the slips — these came up on Tuesday, 9/22.

- **Two of you asked for Question 2** (the template that would not link) and a third asked to go over the slip's problems. That one gets a full walk-through first.
- Why a **for** loop to traverse, but a **while** loop to clear?
- What does a **node** actually *do* in code?
- If we do not free the end of the list, is that a **memory leak**?
- `size() >= capacity()` versus `size_ == capacity_` — any real difference?
- More **analogies**, and more **practice**.
- Several logistics questions — last slide.

Keep writing them

Three people asking the same thing is the clearest signal I get that a slide did not land.

“Please Review Question 2”

Your requests

“Please review question 2.” / “I don't understand question 2, can we review it.” / “Can you go over the problems in this slip please?”

The setup: a class template *declared* in one file and *defined* in another.

```
// Stack.hpp
template <typename T>
class Stack {
public:
    void push(const T& value);
private:
    T data_[100];
    int size_ = 0;
};

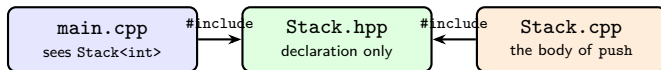
// Stack.cpp
#include "Stack.hpp"
template <typename T>
void Stack<T>::push(const T& v) {
    data_[size_] = v; size_++;
}

// main.cpp
Stack<int> s; s.push(5);
```

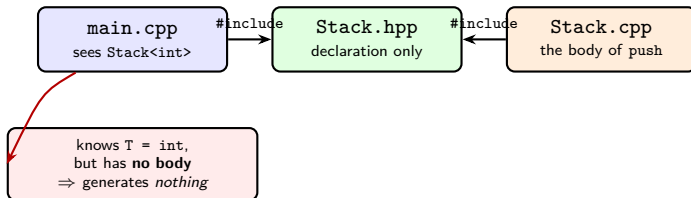
Both files compile. Then the linker says

undefined reference to 'Stack<int>::push(int const&)'

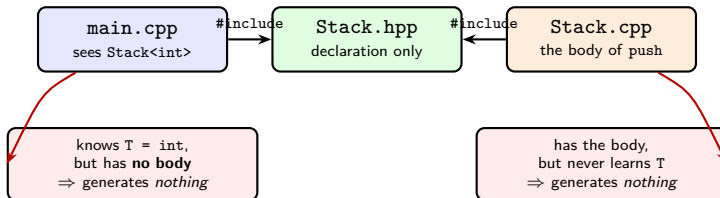
Why It Fails: Follow Each File Separately



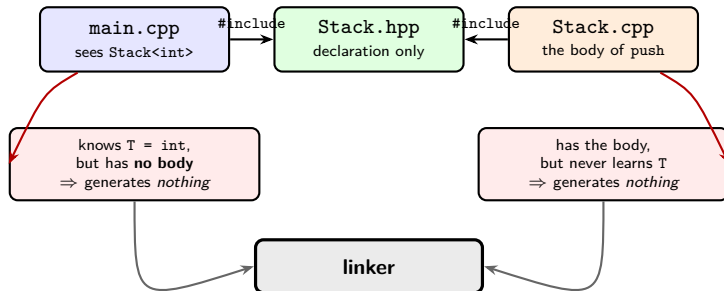
Why It Fails: Follow Each File Separately



Why It Fails: Follow Each File Separately

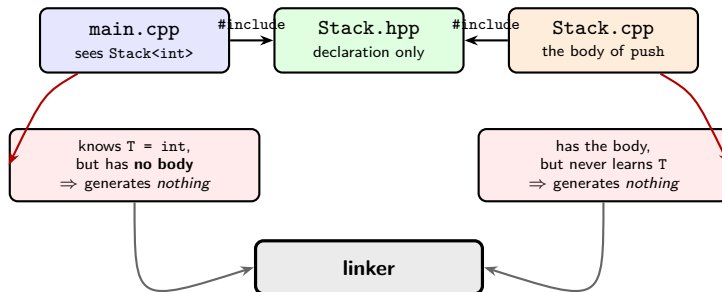


Why It Fails: Follow Each File Separately



*Stack<int>::push was never produced
anywhere — so there is nothing to link to*

Why It Fails: Follow Each File Separately



*Stack<int>::push was never produced
anywhere — so there is nothing to link to*

A template is a **recipe for generating code**. The compiler generates a particular specialization when it has the template definition *and* the needed concrete type. Neither file ever put those two halves together. *The file extension is not the problem. What matters is that the definition is visible at the point where the specialization is needed.*

Question 2: The Fix

The definition must be visible wherever the template is used — so the simplest fix is to write it inside the class:

```
// Stack.hpp -- declaration AND definition, together
template <typename T>
class Stack {
public:
    void push(const T& value) { data_[size_] = value; size_++; }
private:
    T data_[100];
    int size_ = 0;
};
```

Or, to keep the bodies out of the class: put them in `Stack.cpp` and `#include "Stack.cpp"` at the **bottom** of `Stack.hpp` — still “in the header” as far as the compiler is concerned. That is the next slide.

The rule

Ordinary function: declaration in the header, definition in the `.cpp`. **Template:** the definition must be *visible where the template is instantiated* — usually by putting it in the header, or by including a `.cpp` from the header. This is why your `MiniVector` in Project 1B is header-only.

A Complete Working Version: Stack.hpp

```
// Stack.hpp
#ifndef STACK_HPP
#define STACK_HPP
template <typename T>
class Stack {
public:
    void push(const T& value);
    // For simplicity, no pop, top, size, empty methods in this example.

private:
    T data_[100]; // Assume that there are at most 100 elements
                  // in the stack.
    int size_ = 0;
};

// For a template class, including the .hpp file at the end of the
// .hpp file is IMPORTANT.
// Template definitions must be visible when the compiler
// instantiates the template.

#include "Stack.hpp"

#endif
```

The highlighted line is the whole trick, and it has to be *here* — after the class. One `#include "Stack.hpp"` then delivers the declarations *and* the bodies together.

Run all three files: onlinegdb.com/RjHzMve9U (live 9/24/26–10/24/26)

Stack.cpp and main.cpp

```
// Stack.cpp
template <typename T>
void Stack<T>::push(const T& value) {
    data_[size_] = value;
    size_++;
}
```

No include guard, and no #include "Stack.hpp". In this organization, Stack.cpp is not compiled on its own: it is included at the bottom of Stack.hpp, after the class has been declared.

```
// main.cpp
// g++ -std=c++11 main.cpp -o test
#include <iostream>
#include <string>
#include "Stack.hpp"

int main() {
    Stack<int> s;
    s.push(5);

    Stack<std::string> s2;
    s2.push("Hello, how are you?");

    return 0;
}
```

What the compiler actually built

Two separate classes. One with `int data_[100]` and an `int` version of `push`; one with `std::string data_[100]` and a `std::string` version. Neither existed until `main.cpp` named the type — which is the whole reason the bodies had to be visible from here.

“Why for to Traverse, but while to Clear?”

My favourite question of the batch — the answer is about **ordering**, not style.

```
// READING the list: a for loop says everything in one line.  
for (Node* curr = head_; curr != nullptr; curr = curr->next) {  
    std::cout << curr->data << " ";  
}
```

Why for fits here

Start, test and advance are the three things a traversal needs, and the for header holds exactly three slots. It also scopes curr to the loop, so it cannot be misused afterwards.

“Why for to Traverse, but while to Clear?”

My favourite question of the batch — the answer is about **ordering**, not style.

```
// DESTROYING the list, the "symmetric" way. It compiles.  
for (Node* curr = head_; curr != nullptr; curr = curr->next) {  
    delete curr;  
}
```

Read the order of events

The body frees `curr`. *Then* the third slot runs `curr = curr->next`, reading next out of the node that was just freed. Use-after-free, every single iteration.

“Why for to Traverse, but while to Clear?”

My favourite question of the batch — the answer is about **ordering**, not style.

```
// DESTROYING the list, correctly.
while (head_ != nullptr) {
    Node* toDelete = head_;
    head_ = head_->next; // advance BEFORE freeing
    delete toDelete;
}
```

The shape follows the constraint

In a for loop the advance runs *after* the body. Destruction needs the advance *before* the free. The two orders are incompatible, so the loop opens up into a while.

“Why for to Traverse, but while to Clear?”

My favourite question of the batch — the answer is about **ordering**, not style.

And one more difference

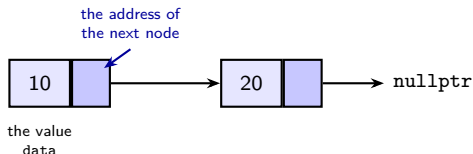
Traversal needs a **separate cursor** (`curr`) because the list must survive unchanged. Clearing has no cursor at all: the loop variable *is* `head_`, and the list itself gets shorter each time round. The initialization and advancement do not fit naturally into a `for` header — the list itself changes as `head_` advances.

Rule of thumb

`for` when you know the three parts up front and the collection does not change. `while` when the loop is consuming the thing it walks.

“What Do Nodes Actually *Do* in Code?”

In our implementation, a node has no behavior of its own. It simply stores two fields:



The work is done by the *links*, not the nodes

In an array, where element $i + 1$ lives is **computed**: $\text{base} + (i+1) * \text{sizeof}(\text{int})$. In a linked list it is **stored**: you read it out of the current node. That one swap — arithmetic for a stored address — is the whole trade-off, and it is why `prepend` is $O(1)$ and `getEntry` is $O(n)$.

Your code only ever does three things with a node: new one, read or write its two fields, delete it.

“If We Don’t Free the End of the List, Is That a Leak?”

Your question

“What happens if we don’t [free] the end of the list — would it be considered a memory leak or something like that, or is it not necessary?”

Yes — and it is necessary. In our linked-list implementation, every node allocated with `new` must eventually be deleted exactly once. Forget the *last* node and you leak one node per list; lose `head_` before walking and you leak *all* of them, with no way to reach them again.

Why you will never spot it by looking

A leaking program prints correct output and exits normally — the operating system reclaims everything when the process ends. Nothing may look wrong in a short-running program, but in a long-running program repeated leaks can steadily increase memory usage. This is why Valgrind exists, and why 10 of Exercise 4’s points are for it.

`size() >= capacity()` vs `size_ == capacity_`

Your question

“`push_back()` in Project 1A has `if (size() >= capacity())`, but in 1B it is `if (size_ == capacity_)`. Is there really any difference functionally? Does this affect inheritance?”

Functionally, no. While the class keeps its own invariant, `size_` never exceeds `capacity_`, so both tests fire at the same instant. The differences are elsewhere:

`>=` versus `==`

`==` trusts the invariant completely: if a bug ever pushes `size_` past `capacity_`, the test is silently skipped and you write off the end of the array. `>=` is slightly more defensive: if a bug has already violated the invariant, it still enters the growth branch. Note that it does not *repair* the broken state — it only avoids compounding it.

Accessors vs raw members — your inheritance question. Good instinct. `size_` and `capacity_` are private, so a **derived class cannot touch them** — only public `size()` and `capacity()`. Code written against the accessors still works in a subclass; code written against the members will not compile there. The accessors are inline, so this costs nothing at run time.

“Could You Use More Analogies?” / “More Practice, Please”

MiniVector: the parking lot

One lot, numbered spaces, all in a row. Finding space 47 is instant — you walk straight to it. But when the lot fills you cannot bolt another row on the end; you rent a bigger lot and **drive every car over**. That relocation is `reserve()`, and it is why one `push_back` is occasionally very expensive while the average stays cheap.

Linked list: the trail of clues

Each clue holds one piece of information plus the address of the next clue. Adding a clue at the front is trivial: write it, point it at the old first clue. But you cannot jump to clue 47 — you follow 46 clues to get there, and you can never walk backwards.

Practice

Exercise 4 is posted today, due Th 10/1: five graded functions — `prepend`, `append`, `insertAt`, `removeValue`, `reverse` — plus an optional sixth, `removeAt`. All of it is today's material. The backup slides at the end of this deck carry the complete code, and the five edge cases are collected just before the reversal section.

Logistics: Textbook, Midterm, Quizzes

You asked	Answer
Which textbook chapters do the lectures map to?	Read Chapter 8 (the List) and Chapter 9 (list implementation, link-based strategies). Note that I do not follow the textbook strictly — I add my own examples and design my own projects, so the lectures are the primary source.
What will the midterm be based on?	Project 1 (MiniVector), Project 2 (Maze Solver), Exercises 1–5, the lecture slips, and the material we discuss in lectures. A guideline will be given before the midterm.
When is the quiz, and what is on it?	There is no formal quiz . I may spring a pop quiz to get answers out of you — those are not counted towards your grade .

Logistics: Slides and Lecture Slips

You asked	Answer
When do the slides go up on Brightspace?	After class, if not sooner — on Brightspace and on the course website.
What does it mean for a slip to be marked <i>late</i> ?	Slips go out 20–30 minutes after class begins . If you arrive after they have been handed out, your attendance is marked late .

What “late” actually costs you

Treat the first few as a warning sign, not a penalty. If a **pattern** develops, I may deduct points from your attendance grade.

If you wrote to me about accommodations

I have your slip and will reply to you directly. It is not too late.

What's Due

Item	Posted	Due
Exercise 4 (Linked List Ops)	today, Th 9/24	Th 10/1
Project 1B: MiniVector	Tu 9/15	Th 10/1

Exercise 4 is posted today

Five graded functions on your own `LinkedList` — `prepend`, `append`, `insertAt`, `removeValue`, `reverse` — and an optional sixth, `removeAt`. Test each one against the five edge cases from this lecture, and run the whole thing under Valgrind on the lab machines: every node you forget to delete is a leak the autograder can see.

Key Takeaways

- **Save before you overwrite.** Every operation today is the same rule: do not destroy the only pointer to something you still need.
- **Prepend is $O(1)$; append is $O(n)$** without a tail pointer, $O(1)$ with one — and the tail is one more invariant to maintain.
- Inserting or deleting in the middle needs the **predecessor**, not the node itself.
- Deleting means **bypass, then delete** — in that order, exactly once.
- The **head is a boundary case**: when there is no predecessor, the pointer to update is head itself rather than a node's next.
- **Reversal** is three pointers, $O(n)$ time, $O(1)$ space, and moves no data at all.

Questions?

See you Tuesday, 9/29 — the Stack ADT and its array-based and linked implementations.

Backup Slides

Complete code, plus one elegant trick.

Backup: All Operations on One Class

```
class LinkedList {
public:
    LinkedList() : head_(nullptr), count_(0) {}
    ~LinkedList();

    void prepend(int value);
    void append(int value);
    bool insertAt(int pos, int value);
    bool removeFront();
    bool removeValue(int target);
    void reverse();

    int size() const { return count_; }
    bool empty() const { return head_ == nullptr; }
    void display() const;

private:
    struct Node { int data; Node* next; };
    Node* head_;
    int count_;
};
```

Backup: Reverse as a Member Function

```
void LinkedList::reverse() {  
    Node* prev = nullptr;  
    Node* curr = head_;  
    while (curr != nullptr) {  
        Node* succ = curr->next;  
        curr->next = prev;  
        prev = curr;  
        curr = succ;  
    }  
    head_ = prev;  
}
```

Note

count_ does not change — reversal does not add or remove nodes. If the class also kept a tail_, reverse() would have to swap head_ and tail_ as well: one more place that invariant has to be maintained.

Backup: Removing the Special Case Entirely

A pointer *to* a pointer lets one loop handle both the head and the interior — this is optional, but worth seeing once:

```
bool removeValue(Node*& head, int target) {  
    Node** link = &head;           // the pointer we may need to change  
    while (*link != nullptr && (*link)->data != target) {  
        link = &((*link)->next); // ... which may be a node's next  
    }  
    if (*link == nullptr) { return false; }  
  
    Node* toDelete = *link;  
    *link = toDelete->next;         // works whether *link is head or next  
    delete toDelete;  
    return true;  
}
```

Why it works

`link` holds the *address of* whichever pointer refers to the candidate node — `&head` at the start, `&(prev->next)` thereafter. Both are just “a `Node*` that must be updated,” so the special case disappears. Not required for Exercise 4.